



UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI SCIENZE AZIENDALI - MANAGEMENT &
INNOVATION SYSTEMS

CORSO DI LAUREA MAGISTRALE IN DATA SCIENCE & GESTIONE
DELL'INNOVAZIONE

Esame di Massive Data Mining

TMDB 5000 Movie Dataset Title Recommendation System

A.A. 2024-2025

Docenti:

Domenico Parente

Luca Rizzuti

Candidata:

DENISE BRANCACCIO - MAT. 0222800163

Indice

1	Introduzione	2
1.1	Strumenti e Tecnologie Utilizzate	2
2	Analisi Esplorativa dei Dati	4
2.1	Pulizia del Dataset	7
2.2	Visualizzazioni Grafiche	8
2.3	Correlazioni	12
2.4	Wordcloud	14
3	Svolgimento del Progetto	15
3.1	Preparazione del Dataset	15
3.1.1	Funzione: clean_json	15
3.1.2	Funzione: cleaning_and_processing	15
3.1.3	Funzione: join_list_elements	16
3.1.4	Caricamento e Pulizia	17
3.2	Sentence Embedding	17
3.3	Calcolo delle Metriche	18
3.3.1	Funzione: calc_sentence_score	18
3.3.2	Funzione: find_genres	18
3.3.3	Funzione: calc_genres_score	19
3.3.4	Funzione: calc_title_score	19
3.3.5	Funzione: calc_people_score	20
3.3.6	Funzione: calc_total_score	21
3.3.7	Ricerca dei Film	21
4	Validazione	22
4.1	Funzione: get_top_k_titles	22
4.2	Validation Set	22
4.3	Funzione: run_validation	25

1 Introduzione

La seguente documentazione illustra il progetto realizzato per l'esame di Massive Data Mining, incentrato sulla creazione di un motore di ricerca per il TMDb 5000 Movie Dataset, il quale raccoglie vaste informazioni su 5000 film diversi includendo:

- Title;
- Genres;
- Overview;
- Cast e Crew;
- Production Companies;
- Keywords;
- Ratings;

L'obiettivo è creare un sistema di raccomandazione sui film in grado di restituire, a partire da una query immessa dall'utente, i 10 film più pertinenti. La query può essere il titolo di un film, scritto in modo approssimativo o corretto, uno o più generi, una trama generale, il nome di un membro del cast o la compagnia di produzione. L'obiettivo principale del progetto è gestire in modo robusto situazioni reali, tenendo conto sia la possibilità che le query siano precise, sia che possano presentare imprecisioni come titoli parziali, errori di battitura, descrizioni molto generiche della trama.

Per realizzare questo lavoro sono state utilizzati due approcci: la cosine similarity, realizzata tramite embedding vettoriali generati dal modello "all-MiniLM-L6-v2" della libreria SentenceTransformers, e la FuzzySearch, metodo della libreria thefuzz basato sulla distanza di Levenshtein, a sua volta basata su quelle di Hamming e Manhattan, ed utilizzato per la ricerca in caso di titoli o nomi parziali.

1.1 Strumenti e Tecnologie Utilizzate

Al fine della realizzazione del progetto è stato utilizzato il linguaggio di programmazione Python tramite un Notebook Jupiter per ottimizzare la pulizia del codice. Sono inoltre state impiegate le seguenti librerie:

- **pandas**: utilizzata per la manipolazione e l'analisi dei dati tabellari tramite DataFrame.
- **ast**: impiegata per convertire rappresentazioni testuali di strutture dati Python (es. liste o dizionari) in oggetti utilizzabili dal programma.
- **numpy**: utilizzata per operazioni numeriche efficienti su array e matrici.
- **thefuzz**: libreria per il *fuzzy matching*, impiegata per misurare la somiglianza tra stringhe anche in presenza di errori o variazioni testuali.
- **sentence-transformers**: utilizzata per generare embedding semantici di frasi e testi mediante modelli basati su Transformer.
- **scikit-learn** (`cosine_similarity`): utilizzata per calcolare la similarità coseno tra vettori, ad esempio tra embedding testuali.

Inoltre per la realizzazione dell'Analisi Esplorativa dei Dati, sono state utilizzate anche le librerie:

- **seaborn**: libreria per la visualizzazione statistica dei dati, utilizzata per creare grafici informativi e facilmente interpretabili.
- **plotly.express**: modulo di Plotly che consente di generare grafici interattivi in modo semplice e rapido.
- **collections** (Counter): struttura dati utilizzata per contare automaticamente le occorrenze degli elementi presenti in una collezione.
- **wordcloud** (WordCloud): libreria utilizzata per generare nuvole di parole, in cui la dimensione di ciascun termine è proporzionale alla sua frequenza nel testo.
- **wordcloud** (STOPWORDS): insieme predefinito di parole comuni da escludere durante la generazione della nuvola di parole, poich poco informative ai fini dell'analisi.

2 Analisi Esplorativa dei Dati

Prima di procedere con le fasi successive dell'analisi, è opportuno svolgere un'Analisi Esplorativa dei Dati, al fine di acquisire una comprensione approfondita del dataset oggetto di studio. Questa fase preliminare consente di esaminare la struttura dei dati, identificare eventuali valori mancanti o anomalie e ottenere una panoramica delle principali caratteristiche delle variabili presenti.

Dopo aver caricato e unito i dataset "tmdb_5000_movies.csv" e "tmdb_5000_credits.csv" tramite l'id univoco dei film, sono state analizzate le sue dimensioni e la composizione delle colonne, verificando il numero di osservazioni e di attributi disponibili, i quali ammontano a 4803 righe e 24 colonne. Tra queste ultime sono presenti: Per una prima esplorazione del dataset è stata utilizzata la funzione `df.describe` di pandas per

Nome Colonna	Descrizione	Tipo di Dato
budget	Investimento per la realizzazione del film in dollari	Intero
genres	Elenco dei generi di ogni film	Lista di dizionari
homepage	URL del sito web ufficiale del film	Stringa
id	Identificativo univoco del film nel database	Intero
keywords	Parole chiave o tag associati ai temi del film	Lista di dizionari
original_language	Lingua originale in cui è stato girato il film	Stringa
overview	Breve trama o sinossi descrittiva del film	Stringa
popularity	Punteggio di popolarità calcolato dalla piattaforma	Float
production_companies	Case di produzione coinvolte nella realizzazione	Lista di dizionari
production_countries	Paesi in cui il film è stato prodotto	Lista di dizionari
release_date	Data di uscita ufficiale del film	Datetime
revenue	Incasso globale generato dal film in dollari	Intero
runtime	Durata totale del film in minuti	Float
spoken_languages	Lingue parlate all'interno del film	Lista di dizionari
status	Stato della produzione	Stringa
tagline	Frase ad effetto o slogan promozionale della pellicola	Stringa
title_x / title_y	Titolo del film	Stringa
vote_average	Voto medio delle recensioni degli utenti (es. da 0 a 10)	Float
vote_count	Numero totale di voti ricevuti dagli utenti	Intero
movie_id	Identificativo del film (chiave usata per il merge)	Intero
cast	Elenco degli attori e dei rispettivi personaggi interpretati	Lista di dizionari
crew	Membri della troupe tecnica (Registi, Sceneggiatori, ecc.)	Lista di dizionari

visualizzare la seguente tabella, contenente i parametri statistici:

- count: Numero di valori non nulli;

- mean: Media aritmetica;
- std: Deviazione Standard;
- min: Valore minimo;
- 25%: Primo quartile Q1;
- 50%: Secondo Quartile Q2;
- 75%: Terzo Quartile Q3;
- max: Valore massimo;

`df.describe()`

	budget	id	popularity	revenue	runtime	vote_average	vote_count	movie_id
count	4.803000e+03	4803.000000	4803.000000	4.803000e+03	4801.000000	4803.000000	4803.000000	4803.000000
mean	2.904504e+07	57165.484281	21.492301	8.226064e+07	106.875859	6.092172	690.217989	57165.484281
std	4.072239e+07	88694.614033	31.816650	1.628571e+08	22.611935	1.194612	1234.585891	88694.614033
min	0.000000e+00	5.000000	0.000000	0.000000e+00	0.000000	0.000000	0.000000	5.000000
25%	7.900000e+05	9014.500000	4.668070	0.000000e+00	94.000000	5.600000	54.000000	9014.500000
50%	1.500000e+07	14629.000000	12.921594	1.917000e+07	103.000000	6.200000	235.000000	14629.000000
75%	4.000000e+07	58610.500000	28.313505	9.291719e+07	118.000000	6.800000	737.000000	58610.500000
max	3.800000e+08	459488.000000	875.581305	2.787965e+09	338.000000	10.000000	13752.000000	459488.000000

Per farlo, calcoliamo innanzitutto la somma dei valori "null" e dei valori presenti per ciascuna colonna, filtrando i risultati in modo da visualizzare solo quelli rilevanti. Ne approfittiamo anche per verificare la presenza di eventuali righe duplicate nel dataset. Infine, per avere un quadro visivo immediato, calcoliamo la percentuale di valori mancanti rispetto al totale e la rappresentiamo tramite un grafico a barre dedicato.

```

1  # Analisi dei valori mancanti e duplicati
2  null_val = df.isnull().sum().sort_values(ascending=False)
3  print("Valori Mancanti:")
4  print(null_val[null_val > 0])
5
6  present_val = df.notnull().sum().sort_values(ascending=False)
7  print("\nValori Presenti:", present_val[present_val > 0])
8  df_plot = pd.DataFrame({
9      'Presenti': present_val,
10     'Mancanti': null_val
11 })
12
13 print("\nDuplicati:", df.duplicated().sum())
14
15 null_perc = (df.isnull().sum() / len(df)) * 100
16 null_perc[null_perc > 0].plot(kind="bar", figsize=(15, 6), title="Percentuale
    dei Valori Mancanti")
17 plt.ylim(0, 100)
18 plt.xticks(rotation=45, ha="right")
19 plt.show()

```

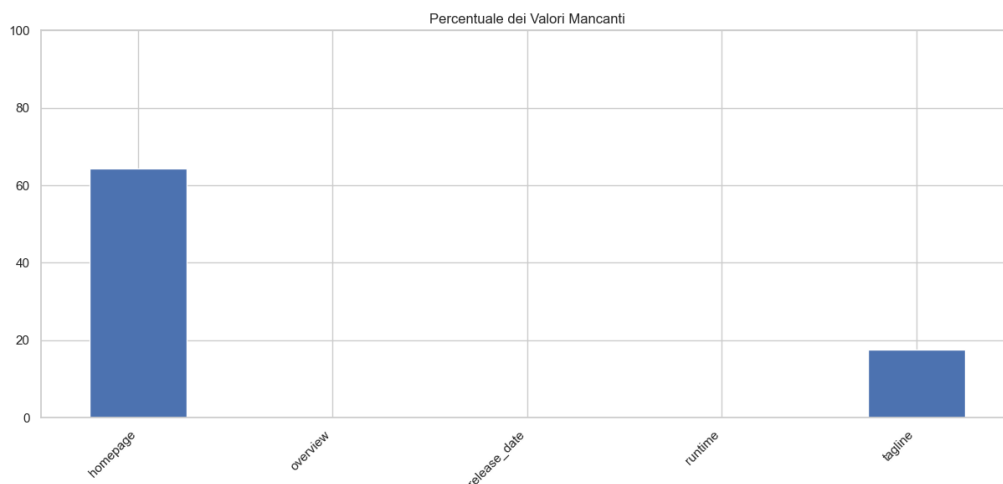
```

Valori Mancanti:
homepage      3091
tagline       844
overview       3
runtime        2
release_date   1
dtype: int64

Valori Presenti: budget      4803
genres            4803
id                4803
keywords          4803
original_title    4803
original_language 4803
production_companies 4803
popularity        4803
cast              4803
title_y           4803
production_countries 4803
revenue           4803
status            4803
spoken_languages  4803
vote_average      4803
title_x           4803
crew              4803
...
homepage          1712
dtype: int64

Duplicati: 0

```



Molte colonne del dataset si presentava originariamente sotto forma di stringhe di testo contenenti liste di dizionari, strutturate in modo simile a un formato JSON. Per rendere queste informazioni facilmente analizzabili, si è reso necessario estrarre solo il dato rilevante, ovvero il nome del genere, dell'attore o della casa di produzione, scartando i codici identificativi e altri metadati non necessari. A tale scopo, è stata definita la funzione personalizzata `clean.json`. Questa funzione effettua innanzitutto un controllo preliminare per verificare che la cella contenga effettivamente del testo e non sia vuota, al fine di evitare anomalie e interruzioni nell'esecuzione del codice. Successivamente, attraverso l'utilizzo di un blocco di gestione degli errori utile a prevenire blocchi causati da eventuali stringhe corrotte, la funzione sfrutta il metodo `literal_eval` della libreria `ast` per convertire in modo sicuro la stringa testuale in una reale struttura dati Python. A questo punto, tramite l'utilizzo di una `list comprehension`, viene effettuata un'iterazione sui dizionari per estrarre esclusivamente il testo associato alla chiave `"name"`. Una volta definita la logica di estrazione, i nomi di tutte le colonne interessate sono stati raccolti all'interno di una lista dedicata. Infine, attraverso un ciclo iterativo e avvalendosi del metodo `.apply()`, la trasformazione è stata applicata in

modo massivo a tutte le variabili selezionate, sovrascrivendo i dati grezzi originali con le nuove liste pulite direttamente all'interno del dataframe.

```
1 # Passaggio da json a lista di stringhe per le colonne specificate
2 def clean_json(json_text):
3     if pd.isna(json_text) or not isinstance(json_text, str):
4         return []
5
6     try:
7
8         converted_list = ast.literal_eval(json_text)
9         return [item["name"] for item in converted_list if "name" in item]
10    except (ValueError, SyntaxError):
11        return []
12
13    colonne_json = [
14        "genres",
15        "keywords",
16        "production_companies",
17        "production_countries",
18        "spoken_languages",
19        "cast",
20        "crew"
21    ]
22    for col in colonne_json:
23        df[col] = df[col].apply(clean_json)
24    df.head()
```

2.1 Pulizia del Dataset

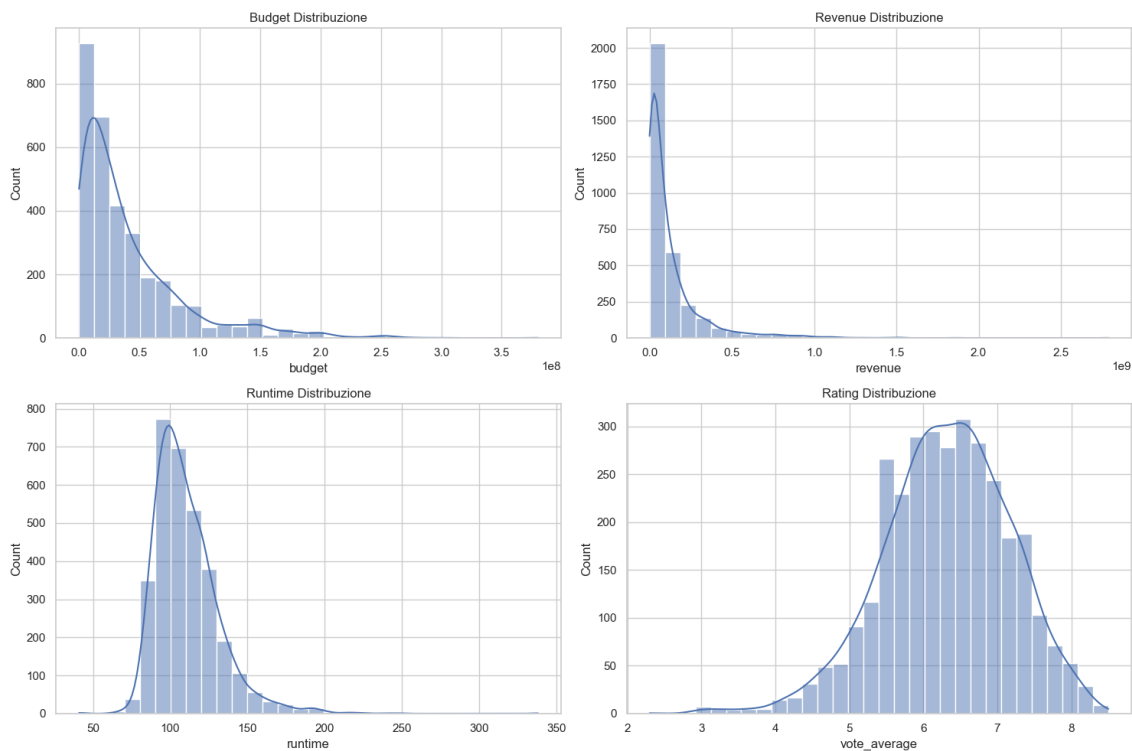
Al fine di preparare il dataset per le successive esplorazioni, è stata eseguita una fase di pulizia e trasformazione dei dati. Inizialmente, la variabile relativa alla data di uscita è stata convertita in un formato temporale adeguato, gestendo in modo sicuro eventuali errori di formattazione e permettendo l'estrazione dell'anno di pubblicazione all'interno di una nuova colonna dedicata. Successivamente, per snellire la struttura delle informazioni, sono state rimosse alcune variabili ritenute non necessarie per lo scopo dell'analisi, come gli identificativi ridondanti, gli slogan promozionali, lo stato della produzione e i collegamenti alle pagine web. Si è poi proceduto all'eliminazione dei record privi del dato sulla durata della pellicola e alla normalizzazione del nome della colonna del titolo, correggendo le alterazioni dovute alla precedente unione dei dati. Per garantire la significatività e l'affidabilità delle valutazioni finanziarie e qualitative, il dataframe è stato rigorosamente filtrato, conservando esclusivamente i film caratterizzati da budget, incassi e votazioni medie strettamente maggiori di zero. Infine, sfruttando i dati economici appena purificati, è stata generata una nuova metrica utile a calcolare il ritorno sull'investimento (ROI) in formato percentuale, indicatore fondamentale per misurare in modo standardizzato la profittabilità economica di ciascuna opera cinematografica.

```
1 # Pulizia e trasformazione dei dati
2 df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")
3 df["year"] = df["release_date"].dt.year.astype("Int64")
4 df = df.drop(["title_y", "tagline", "homepage", "movie_id", "status"], axis =
5             1)
6 df = df.dropna(subset=["runtime"])
7 df = df.rename(columns={"title_x": "title"})
8 df = df[(df["budget"] > 0) & (df["revenue"] > 0) & (df["vote_average"] > 0)]
9 df["roi"] = (df["revenue"] - df["budget"]) / df["budget"] * 100
```

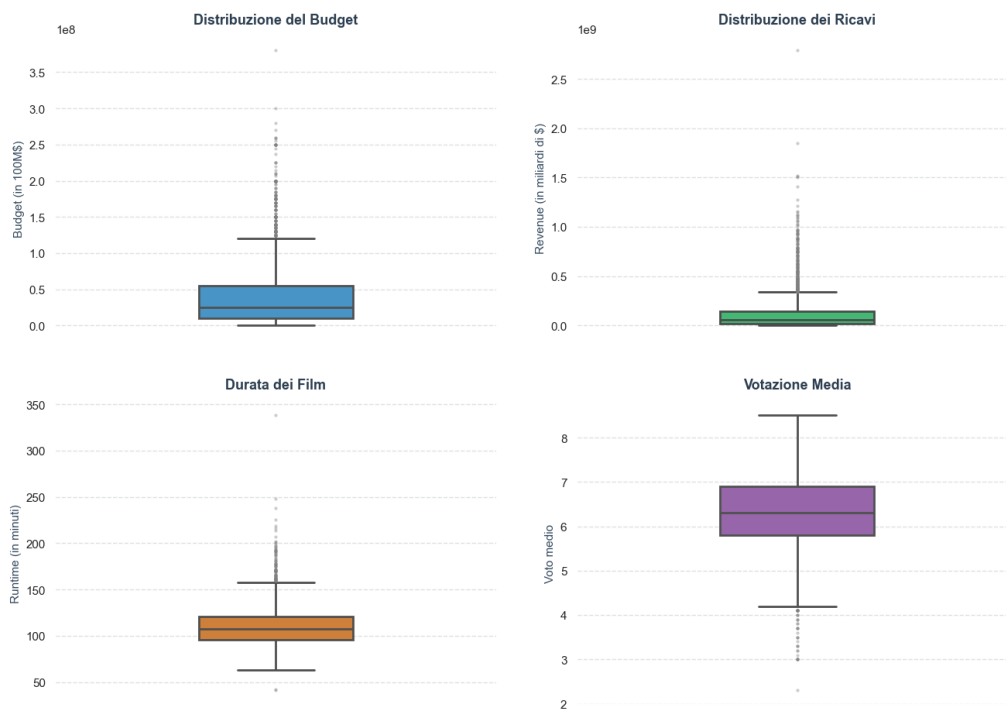
2.2 Visualizzazioni Grafiche

Per approfondire la comprensione delle variabili a seguito della pulizia sono stati realizzati una serie di grafici a istogramma. Tali visualizzazioni consentono di esaminare la distribuzione, la dispersione e la frequenza dei dati relativi al budget, alla revenue, al runtime e ai ratings.

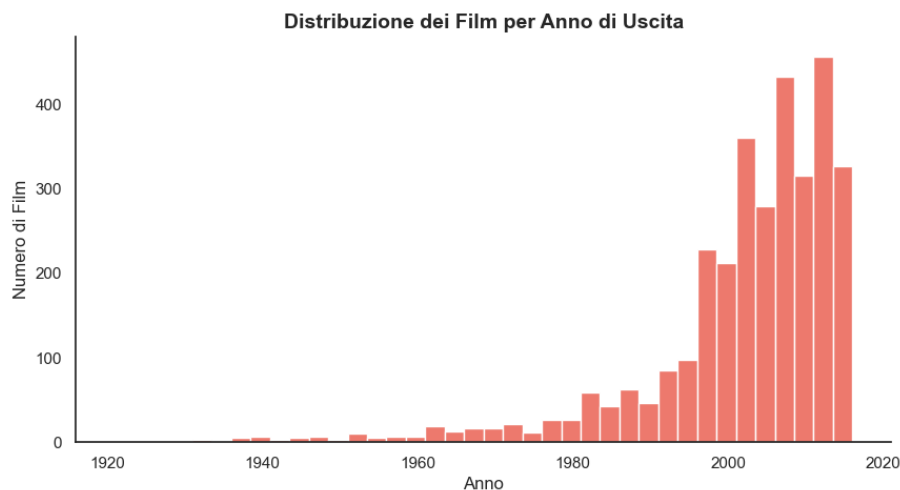
- Budget: Forte asimmetria positiva, il 75% dei film ha un budget inferiore ai 40 milioni di dollari. Sulla destra si estende una coda sottile e allungata che rappresenta i rari blockbuster milionari;
- Revenue: Forte asimmetria positiva, più estrema rispetto al grafico precedente. Il 75% dei film incassa meno di 93 milioni, ma presenta una coda molto lunga grazie al film Avatar, campione di incassi con 2,8 miliardi di dollari;
- Runtime: Classica distribuzione simile a una normale, con un picco centrale tra i 90 e 120 minuti, prevedibile da una media di 107. Ai lati di questo picco centrale le barre scendono mostrando pochissimi cortometraggi o film estesi;
- Vote Average: Distribuzione normale leggermente spostata a destra con voti molto bassi e che mostra la presenza di alcuni film ancora non valutati;



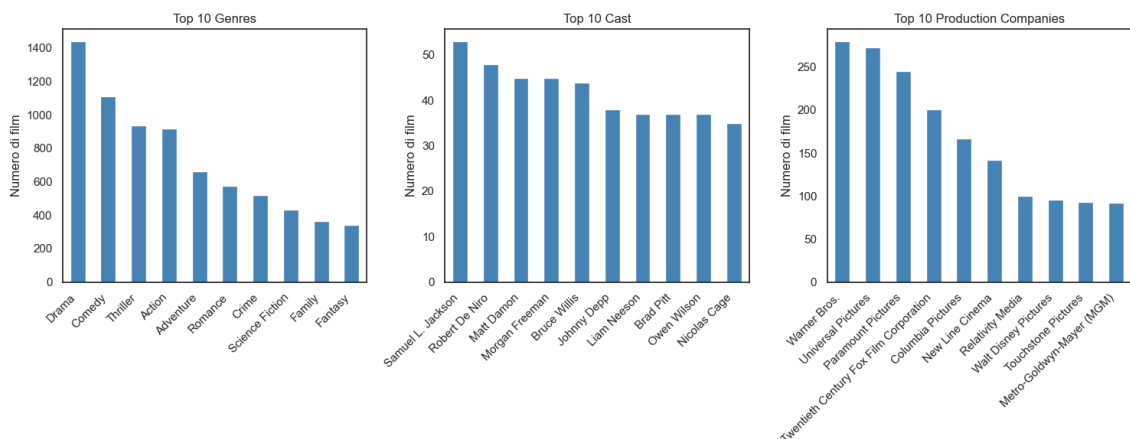
Successivamente all'analisi delle distribuzioni, l'esplorazione visiva è proseguita con l'utilizzo di boxplot per le principali variabili numeriche. Questa tipologia di grafico offre una sintesi visiva estremamente efficace per valutare la dispersione dei dati, mettendo in evidenza la mediana, la variabilità e l'intervallo in cui si concentra la maggior parte dei valori. L'obiettivo principale di questa rappresentazione è l'identificazione degli outlier, ovvero quei valori anomali che si discostano in modo significativo dal corpo principale della distribuzione. Nel contesto cinematografico, l'osservazione dei boxplot permette di individuare e isolare facilmente le eccezioni statistiche, come i grandi blockbuster caratterizzati da investimenti o incassi straordinariamente elevati rispetto alla media del settore, oppure pellicole con durate e valutazioni estreme. Riconoscere e mappare queste anomalie statistiche risulta fondamentale per comprendere la reale eterogeneità del dataset e per valutare in che misura tali eccezioni possano influenzare i modelli e le elaborazioni successive.



Attraverso la creazione di un grafico a barre, è stata possibile l'osservazione della tendenza storica, la quale mette in luce una presenza piuttosto marginale di pellicole risalenti ai primi decenni del Novecento, seguita da una crescita lenta e graduale che prende il via intorno agli anni Ottanta. Il dato visivo più rilevante emerge tuttavia a partire dalla fine degli anni Novanta, momento in cui si registra un'impennata quasi esponenziale della produzione cinematografica registrata, fino a raggiungere il picco massimo intorno alla metà degli anni 2010.



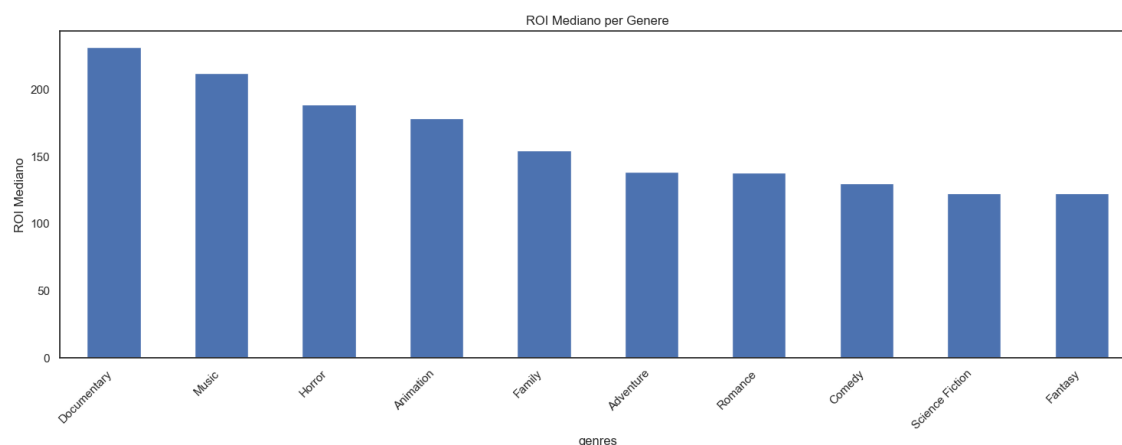
Per arricchire l'analisi anche delle variabili di natura categoriale sono stati creati tre grafici a barre affiancati, isolando e quantificando la classifica delle prime 10 posizioni relative ai generi, ai membri del cast e alle case di produzione ordinati in base al numero complessivo di pellicole in cui compaiono. Il primo grafico, dedicato ai generi cinematografici, evidenzia una netta predominanza dei film drammatici e delle commedie, seguiti a distanza dai thriller e dai film d'azione. Il secondo grafico sposta l'attenzione sulle presenze attoriali, mostrando in modo chiaro i volti più assidui all'interno del panorama analizzato, tra cui Samuel L. Jackson, Robert De Niro e Matt Damon con rispettivamente la presenza in 52, 47 e 45 film. L'ultimo grafico illustra invece il peso delle diverse case di produzione, confermando l'assoluto predominio e la forte presenza storica delle case di Hollywood quali Warner Bros, Universal Pictures e Paramount Pictures.



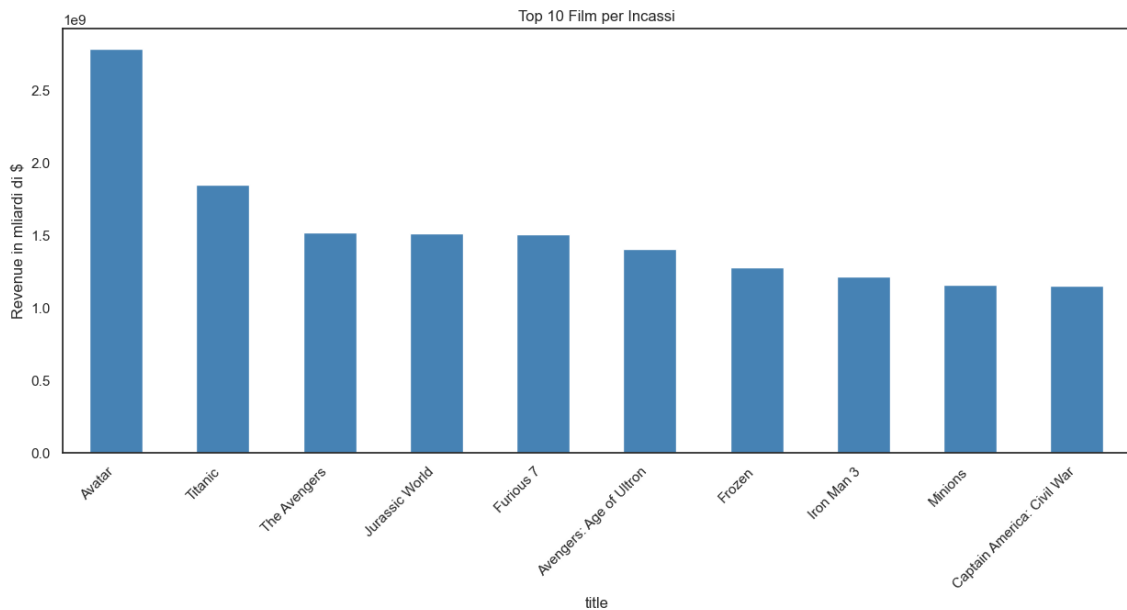
1	Generi totali: 19	
2	Top 10 generi:	
3	Drama	1439
4	Comedy	1110
5	Thriller	935
6	Action	918

7	Adventure	661
8	Romance	574
9	Crime	520
10	Science Fiction	431
11	Family	365
12	Fantasy	342

Il seguente grafico si focalizza invece sul ritorno sull'investimento mediano in funzione del genere. Dall'osservazione di questa metrica emerge che le pellicole in grado di garantire in media i ritorni percentuali più elevati sono i documentari, seguiti a breve distanza dalle opere musicali e dai film horror. Questa spiccata efficienza economica tipicamente riconducibile ai costi di produzione molto più contenuti che caratterizzano queste produzioni. Al contrario, generi che richiedono per natura budget più elevati per effetti visivi e campagne marketing, come fantascienza e fantasy, si posizionano alla fine di questa classifica.



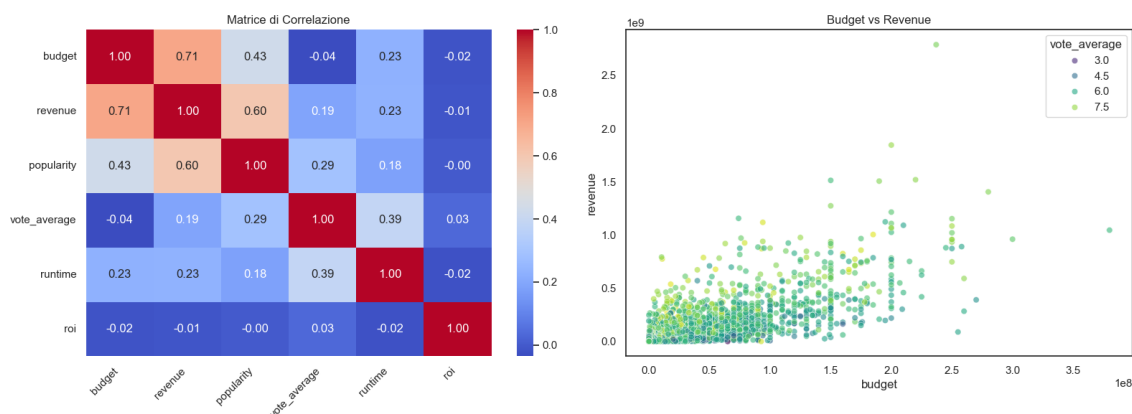
Al fine di identificare i maggiori successi commerciali, è stato predisposto un grafico a barre che illustra i primi dieci film per volume di incassi globali. La visualizzazione evidenzia il netto primato isolato di Avatar, l'unica opera a sfiorare i 2,8 miliardi di dollari, seguita a debita distanza da Titanic. Il resto della graduatoria risulta invece fortemente dominato da noti franchise e blockbuster seriali che si attestano tutti stabilmente sopra la significativa soglia del miliardo di dollari di ricavi, delineando chiaramente i titoli a più alta redditività storica all'interno del campione.



2.3 Correlazioni

Per approfondire le relazioni lineari tra le metriche quantitative del dataset, la matrice di correlazione offre una quantificazione precisa dei legami statistici attraverso i rispettivi coefficienti numerici. Emerge in modo evidente una forte associazione positiva, pari a 0.71, tra il budget stanziato e i ricavi generati, a dimostrazione del fatto che investimenti finanziari più cospicui tendono a tradursi in maggiori incassi al botteghino. Si rileva inoltre una significativa correlazione positiva tra gli incassi e l'indice di popolarità (0.60), suggerendo che il successo commerciale sia strettamente connesso alla risonanza mediatica dell'opera. Al contrario, la variabile del ritorno sull'investimento presenta valori stabilmente prossimi allo zero con tutte le altre metriche, confermando analiticamente che la redditività percentuale prescinde dalla scala economica assoluta del progetto cinematografico.

Questo scenario trova un riscontro visivo diretto nel grafico a dispersione adiacente, in cui la distribuzione della nuvola di punti si sviluppa diagonalmente verso l'alto, convalidando la tendenza alla crescita dei ricavi in concomitanza con l'aumento dei costi di produzione. L'integrazione di una scala cromatica legata alla votazione media arricchisce la lettura del grafico con una dimensione qualitativa, evidenziando come le pellicole maggiormente apprezzate dal pubblico, contraddistinte dalle tonalità più chiare e accese, tendano a concentrarsi prevalentemente nelle fasce di incasso medio-alte, pur non escludendo la presenza di produzioni a basso budget capaci di ottenere eccellenti valutazioni critiche.

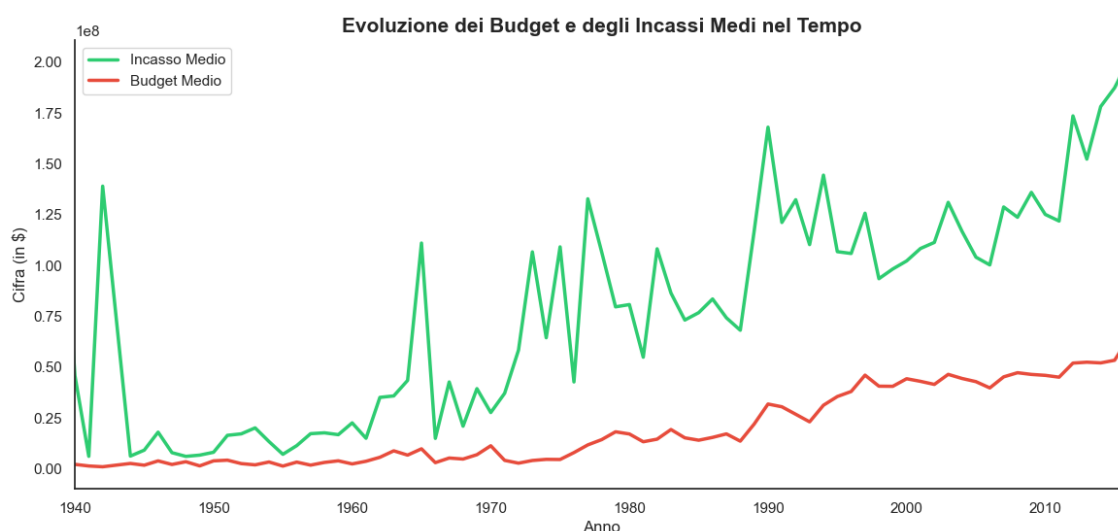


Per tracciare un quadro storico dell'evoluzione finanziaria del settore cinematografico, è stato sviluppato un grafico a linee che pone a confronto l'andamento del budget medio e dell'incasso medio nel corso dei decenni.

La curva dedicata al budget medio (in rosso) mostra una stabilità quasi assoluta dal 1940 fino alla fine degli anni Settanta, mantenendosi su livelli storicamente molto contenuti. A partire dagli anni Ottanta, tuttavia, l'investimento medio intraprende una crescita costante e progressiva che accelera sensibilmente nel nuovo millennio, riflettendo chiaramente l'avvento dei moderni blockbuster e il generale aumento dei costi di produzione e di marketing.

Al contrario, la linea degli incassi medi (in verde) rivela una forte volatilità nei primi quarant'anni della linea temporale. Questa spiccata oscillazione, caratterizzata da picchi isolati e bruschi cali, è legata con ogni probabilità alla scarsità di dati per i film più datati, dove il successo straordinario di una singola pellicola era in grado di alterare drasticamente la media del rispettivo anno.

L'elemento analitico più rilevante si osserva a partire dagli anni Novanta: pur mantenendo una forte componente fluttuante, la curva dei ricavi sperimenta un'impennata notevole, distanziando in modo sistematico quella dei costi. Il progressivo ampliamento della forbice tra entrate e uscite nell'ultimo ventennio evidenzia la straordinaria capacità del mercato cinematografico contemporaneo di massimizzare i profitti su scala globale, portando gli incassi medi verso il picco storico in prossimità della fine della linea temporale.



2.4 Wordcloud

Per esplorare le tematiche ricorrenti e i nuclei narrativi che caratterizzano le produzioni cinematografiche del dataset, è stata realizzata una wordcloud focalizzata sui termini più frequenti all'interno delle trame dei film. L'analisi visiva mette immediatamente in luce i motori concettuali ed emotivi universali del cinema: parole di grandi dimensioni come "life", "find", "world" e "family" indicano chiaramente come la stragrande maggioranza delle sceneggiature si sviluppino attorno a percorsi di ricerca, dinamiche relazionali e contesti esistenziali o d'azione di ampio respiro.

Accanto a questi pilastri macrotematici, si nota una forte concentrazione di termini legati ai legami interpersonali e agli archetipi dei personaggi, quali "man", "young", "father", "friend" e "love" che testimoniano la centralità dei conflitti familiari e delle evoluzioni sentimentali nelle storie. Infine, la ricorrenza di marcatori di contesto come "time", "new", "town" e "home" delinea la cornice spaziale e temporale tipica in cui prendono vita le vicende. Nel complesso, la visualizzazione restituisce una mappatura semantica coerente, confermando come l'industria cinematografica si affidi a tropi e concetti universali per garantire l'immediata immedesimazione e l'efficacia comunicativa sul pubblico globale.

[illegible]

3.1 Preparazione del Dataset

L'avvio del progetto ha richiesto una cruciale fase preliminare di pulizia e pre-elaborazione dei dati, mirata a normalizzare la struttura delle variabili categoriali più complesse presenti nel dataset, come i generi cinematografici, i membri del cast e le case di produzione. Molte di queste informazioni, infatti, si presentavano originariamente sotto forma di stringhe testuali contenenti strutture dati nidificate e formattate in modo simile a frammenti JSON. Per poter manipolare e quantificare agevolmente queste informazioni, si è reso necessario sviluppare una specifica funzione di parsing denominata `clean.json`. Sfruttando il modulo `ast` (Abstract Syntax Trees) per convertire in sicurezza la stringa in una struttura nativa Python, la funzione analizza la lista di dizionari così ottenuta ed estrae da ciascun elemento esclusivamente il valore associato alla chiave utile, in questo caso il nome dell'attributo. Questo processo di estrazione consente di trasformare blocchi di testo complessi in semplici liste di stringhe pulite, ponendo le basi strutturali fondamentali per tutte le successive analisi statistiche e visualizzazioni grafiche.

A valle della definizione della logica di estrazione testuale, il processo di preparazione dei dati si concretizza nella strutturazione di una pipeline complessiva di pulizia e pre-elaborazione, racchiusa all'interno della funzione `cleaning_and_preprocessing`. L'operazione iniziale di questa routine mira a garantire la solidità

del dataset attraverso la rimozione automatica di tutti i record contenenti valori mancanti o nulli. Successivamente, l'attenzione si sposta sulla standardizzazione delle informazioni temporali: la colonna relativa alla data di uscita delle pellicole viene prima forzata in un formato data coerente (gestendo eventuali errori di conversione) e subito dopo riorganizzata visivamente secondo lo standard europeo basato su giorno, mese e anno.

Dopo aver risolto alcune ridondanze di nomenclatura, come la rinomina della colonna del titolo per una maggiore chiarezza formale, il cuore della funzione entra in gioco applicando massivamente la precedente routine `clean_json` a tutte le variabili complesse. Questo passaggio permette di estrarre e appiattire in modo automatico le informazioni contenute in colonne fondamentali quali i generi, il cast, la troupe, le parole chiave, le case di produzione e le lingue parlate. Nel caso specifico dei generi cinematografici, viene integrata un'ulteriore accortezza per l'omogeneità dei dati, convertendo ogni termine estratto in minuscolo per prevenire discrepanze durante le future fasi di raggruppamento. Infine, la pipeline di lavoro si conclude con una fase di snellimento strutturale: tramite l'eliminazione mirata di quelle colonne ritenute ridondanti o non in linea con i successivi obiettivi di analisi (come gli identificativi numerici, i titoli duplicati e alcune variabili geografiche o finanziarie non necessarie in questo step), la funzione restituisce un dataframe perfettamente pulito, compatto e pronto per essere esplorato statisticamente.

```

1  def cleaning_and_preprocessing(df):
2      df.dropna(inplace=True)
3      df["release_date"] = pd.to_datetime(df["release_date"], errors='coerce')
4      df["release_date"] = df["release_date"].dt.strftime('%d/%m/%Y')
5      df.rename(columns={"title_x": "title"}, inplace=True)
6      df["genres"] = df["genres"].apply(clean_json)
7      df["genres"] = df["genres"].apply(lambda x: [genere.lower() for genere in
8          x])
9      df["keywords"] = df["keywords"].apply(clean_json)
10     df["cast"] = df["cast"].apply(clean_json)
11     df["crew"] = df["crew"].apply(clean_json)
12     df["production_companies"] = df["production_companies"].apply(clean_json)
13     df["production_countries"] = df["production_countries"].apply(clean_json)
14     df["spoken_languages"] = df["spoken_languages"].apply(clean_json)
15     df.drop(["id", "movie_id", "title_y", "budget", "revenue",
16         "production_countries"], axis=1, inplace=True)
17     return df

```

3.1.3 Funzione: `join_list_elements`

Come ultimo passaggio della fase di preparazione e formattazione dei dati, si è resa necessaria un'operazione di consolidamento delle variabili precedentemente estratte. A tale scopo è stata implementata la funzione `join_list_elements`, concepita per trasformare le strutture a lista in formati testuali lineari e immediatamente leggibili. La routine accetta in ingresso il dataframe e un elenco specifico di colonne su cui operare, iterando su ciascuna di esse in modo sistematico. Attraverso l'applicazione di una funzione lambda, il codice verifica dinamicamente la natura di ogni singolo elemento: qualora si tratti di una lista (come nel caso dei molteplici generi, degli attori o delle parole chiave associati a una singola pellicola), i suoi componenti vengono concatenati in un'unica stringa continua, separati da una virgola e da uno spazio; in caso contrario, il dato viene convertito in modo precauzionale in un formato testuale standard. Questa trasformazione finale risulta strategica non solo per migliorare la leggibilità diretta e l'esplorazione visiva del dataset, ma anche per predisporre le variabili categoriali a eventuali esportazioni in formato tabellare o a successive elaborazioni algoritmiche basate sull'analisi del testo, garantendo la massima compatibilità e coerenza strutturale.

```

1  def join_list_elements(dataframe, columns):

```

```

2         for col in columns:
3             dataframe[col] = dataframe[col].apply(
4                 lambda x: ", ".join(x) if isinstance(x, list) else str(x)
5             )
6         return dataframe

```

3.1.4 Caricamento e Pulizia

A completamento della fase di preparazione, il blocco principale del codice orchestra l'integrazione iniziale e il salvataggio dei dati. I due dataset nativi relativi a pellicole e crediti vengono caricati e fusi in un unico dataframe coerente attraverso il raccordo dei rispettivi identificativi numerici. Su questa struttura unificata viene poi applicata la pipeline di pulizia descritta in precedenza; infine, il dataset normalizzato viene esportato in un nuovo file CSV, garantendo la persistenza di un archivio pulito e già pronto per le successive fasi di analisi statistica.

```

1     movies = pd.read_csv("Dataset/tmdb_5000_movies.csv")
2     credits = pd.read_csv("Dataset/tmdb_5000_credits.csv")
3     df = pd.merge(movies, credits, left_on="id", right_on="movie_id")
4
5     df = cleaning_and_preprocessing(df)
6     df.to_csv('tmdb_5000_pulito.csv', index=False)
7     print("Dataset salvato con successo!")

```

3.2 Sentence Embedding

Questa sezione del progetto avvia la fase di modellazione e arricchimento delle caratteristiche (feature engineering), finalizzata a trasformare il patrimonio informativo testuale in elementi computabili dall'algoritmo. Inizialmente, la funzione `join_list_elements` viene applicata per consolidare in stringhe lineari i metadati relativi a parole chiave, cast, troupe e case cinematografiche. Parallelamente, lo script estrae e ordina i generi univoci del dataset e genera una colonna accessoria contenente l'acronimo in minuscolo di ciascun titolo, utile per supportare logiche di ricerca rapida.

Il passaggio chiave per l'analisi semantica consiste nella creazione di una variabile testo omnicomprensiva, denominata `sentence_var`. Questa stringa unisce tutti i descrittori di un film: titolo, trama (overview), parole chiave, slogan (tagline), generi, attori, membri chiave della troupe e produttori. Questo blocco unico di testo viene infine dato in pasto al modello `SentenceTransformer` che, tramite il metodo `encode`, ne esegue la vettorializzazione, restituendo una matrice di embedding pronti per il calcolo della similarità.

Nel campo dell'elaborazione del linguaggio naturale, il sentence embedding rappresenta il processo di mappatura di stringhe di testo di lunghezza variabile (frasi, paragrafi o interi documenti) in vettori numerici densi a dimensione fissa.

A differenza dei modelli classici come la Bag-of-Words o TF-IDF che si limitano a contare la presenza delle parole creando matrici sparse e prive di senso logico, i sentence embedding sono progettati per catturare il contesto e il significato semantico. L'obiettivo fondamentale è fare in modo che la distanza geometrica tra due vettori nello spazio multidimensionale rifletta la somiglianza concettuale dei testi di partenza. Di conseguenza, due film che descrivono trame simili utilizzando sinonimi o vocaboli completamente diversi saranno comunque mappati in punti molto vicini dello spazio vettoriale, consentendo di superare i limiti della rigida ricerca per parole chiave.

```

1 df = join_list_elements(df, ["keywords", "cast", "crew",
2                             "production_companies"])
3
4 all_genres = [genres for sublist in df["genres"] for genres in sublist]
5 unique_genres = sorted(list(set(all_genres)))
6 #print(f"Generi individuati:{unique_genres}")
7
8 df["acronym"] = df["title"].str.lower().str.split().apply(
9     lambda parole: " ".join([parola[0] for parola in parole] if isinstance(parole,
10                                     list) else "")
11 )
12 #print(df[["title", "acronym"]].head())
13
14 genres_str = df["genres"].apply(lambda x: " ".join(x) if isinstance(x, list)
15                                 else "")
16
17 sentence_var = (df["title"] + " " + df["overview"] + " " + df["keywords"] + "
18                 " + df["tagline"] + " " + genres_str + " " + df["cast"] + " " + df["crew"]
19                 + " " + df["production_companies"])
20 model = SentenceTransformer('all-MiniLM-L6-v2')
21 embeddings_sentence = model.encode(sentence_var.tolist(),
22                                   convert_to_numpy=True)

```

3.3 Calcolo delle Metriche

3.3.1 Funzione: calc_sentence_score

Per tradurre l'input testuale dell'utente in un valore metrico direttamente computabile, è stata implementata la funzione `calc_sentence_score`. Questa routine riceve la query espressa in linguaggio naturale e, come prima operazione, la converte in un vettore denso sfruttando il medesimo modello di Sentence Transformer utilizzato per l'indicizzazione del catalogo cinematografico, garantendo così la coerenza dello spazio vettoriale. Successivamente, viene calcolata la cosine similarity tra l'embedding della richiesta e la matrice precomputata di tutte le pellicole, determinando la vicinanza geometrica e quindi l'affinità concettuale tra i testi. Poiché questo coefficiente statistico oscilla originariamente all'interno di un intervallo compreso tra -1 e 1, il codice introduce un passaggio cruciale di normalizzazione attraverso la tecnica del Min-Max scaling. Riscaldando matematicamente i vettori in modo che la minor somiglianza rilevata corrisponda a 0 e la maggiore a 1, la funzione restituisce un vettore di punteggi standardizzato e perfettamente calibrato.

```

1 def calc_sentence_score(user_query):
2     query_embedding = model.encode([user_query], convert_to_numpy=True)
3     sentence_cosine = cosine_similarity(query_embedding,
4                                         embeddings_sentence)[0]
5
6     sentence_score_normalized = (sentence_cosine - sentence_cosine.min()) /
7     (sentence_cosine.max() - sentence_cosine.min())
8     return sentence_score_normalized

```

3.3.2 Funzione: find_genres

Al fine di intercettare in modo mirato le preferenze categoriali espresse esplicitamente dall'utente, è stata sviluppata la funzione `find_genres`. Questa routine esegue una scansione della richiesta testuale per verifi-

care la presenza diretta di specifici generi cinematografici all'interno del linguaggio naturale. Come prima operazione, l'input viene interamente normalizzato in caratteri minuscoli per garantire la totale insensibilità alle maiuscole ed evitare mancate corrispondenze. Successivamente, la funzione confronta la stringa così elaborata con l'elenco dei generi univoci precedentemente mappati nel dataset, estraendo ogni termine che compaia come sottostringa all'interno della richiesta. Questo meccanismo di filtraggio basato su parole chiave esplicite risulta di fondamentale importanza strategica per il motore di ricerca, in quanto permette di isolare i desiderata categorici dell'utente prima delle successive fasi algoritmiche. In questo modo, il sistema ottiene un elenco pulito di generi di riferimento che può essere utilizzato per applicare filtri booleani rigidi o logiche di potenziamento dei punteggi, affiancando alla comprensione puramente semantica degli embedding un controllo tassonomico preciso e deterministico.

```
1 def find_genres(user_query, unique_genres):
2     query_lowered = user_query.lower()
3     genres_found = [gen for gen in unique_genres if gen in query_lowered]
4     return genres_found
```

3.3.3 Funzione: calc_genres_score

Per valorizzare la congruenza tassonomica tra le richieste espresse dall'utente e il catalogo a disposizione, è stata implementata la funzione `calc_genres_score`. Questa routine ha il compito di quantificare l'affinità categoriale assegnando un punteggio matematico basato sulla sovrapposizione dei generi cinematografici. Il codice cicla attraverso tutte le pellicole del dataframe e valuta lo scenario di partenza: qualora l'elenco dei generi estratti dalla query risulti vuoto, viene assegnato di default un punteggio pari a zero a ogni riga, neutralizzando l'impatto di questa metrica. Nel caso in cui l'utente abbia invece specificato uno o più generi di interesse, la funzione calcola l'intersezione logica tra l'insieme dei desiderata e l'insieme dei generi associati al singolo film. Questo valore viene successivamente normalizzato dividendolo per il numero totale di generi richiesti dall'utente, generando così un indice standardizzato compreso tra 0 e 1 che rappresenta la percentuale esatta di soddisfacimento dei requisiti. Questo punteggio deterministico agisce come un moltiplicatore di rilevanza all'interno del motore complessivo, permettendo al sistema di premiare e spingere verso l'alto i titoli che rispettano in modo rigoroso i vincoli categorici indicati esplicitamente nella query.

```
1 def calc_genres_score(query_genres):
2     genres_score = []
3     for genres in df["genres"]:
4         if len(query_genres) == 0:
5             genres_score.append(0)
6         else:
7             intersection = set(query_genres) & set(genres)
8             num_genres = len(intersection) / len(query_genres)
9             genres_score.append(num_genres)
10    return genres_score
```

3.3.4 Funzione: calc_title_score

Per integrare la ricerca diretta dei titoli all'interno della logica di raccomandazione, la funzione `calc_title_score` implementa una strategia di corrispondenza multi-livello che valuta l'affinità tra la richiesta dell'utente e il catalogo. Dopo aver normalizzato la query in caratteri minuscoli, l'algoritmo calcola in parallelo tre indici

di somiglianza, estraendo per ciascun film il valore massimo ottenuto: una corrispondenza esatta, una concordanza deterministica con gli acronimi e un punteggio testuale di tipo fuzzy. Quest'ultimo approccio si fonda sulla Distanza di Levenshtein, una metrica matematica che quantifica il numero minimo di modifiche (inserimenti, cancellazioni o sostituzioni) necessarie per trasformare una stringa in un'altra. Superando la rigidità della logica booleana dicotomica, la ricerca fuzzy valuta il grado di somiglianza, tollerando variazioni o lievi errori di battitura. Nello specifico, l'utilizzo della variante `partial_ratio` ottimizza il confronto tra testi di lunghezze differenti, verificando se la query, tipicamente più corta, corrisponde in modo ottimale a una specifica porzione del titolo del film. L'unione di queste tre metriche garantisce un motore di riconoscimento robusto e flessibile, capace di individuare l'opera desiderata anche a fronte di imprecisioni o acronimi.

```

1  def calc_title_score(user_query, df):
2      query_normalized = user_query.lower()
3      titoli_lower = df["title"].str.lower()
4
5      exact_match = titoli_lower.str.contains(query_normalized,
6                                              regex=False).astype(float)
7
8      acronym_match = (df["acronym"] == query_normalized).astype(float)
9
10     fuzzy_match = titoli_lower.apply(lambda x:
11                                     fuzz.partial_ratio(query_normalized, x) / 100.0)
12
13     best_title_score = np.maximum.reduce([exact_match, acronym_match,
14                                           fuzzy_match])
15     return best_title_score

```

3.3.5 Funzione: `calc_people_score`

Per intercettare i riferimenti a specifiche figure umane o societarie menzionate dall'utente, la funzione `calc_people_score` estende la logica di confronto testuale ai metadati relativi al cast, alla troupe cinematografica e alle case di produzione. Il codice unifica inizialmente queste tre informazioni in un unico blocco di testo normalizzato in caratteri minuscoli, azzerando l'influenza delle maiuscole. Su questa stringa combinata vengono calcolati in parallelo due distinti indici di somiglianza rispetto alla query: una verifica di corrispondenza esatta, che rileva la presenza diretta del nome cercato e un confronto di tipo fuzzy basato sempre sulla metrica del `partial_ratio`. Quest'ultimo strumento si rivela essenziale per identificare correttamente registi, attori o marchi editoriali anche in presenza di refusi, omissioni o grafie parziali di nomi complessi. Attraverso l'applicazione della funzione `np.maximum`, il sistema seleziona infine il valore più elevato tra i due punteggi ottenuti per ciascuna pellicola, garantendo una valutazione ottimale dell'affinità e fornendo al motore di raccomandazione una metrica robusta per premiare i titoli legati ai professionisti del cinema desiderati dall'utente.

```

1  def calc_people_score(user_query, df):
2      query_normalized = user_query.lower()
3      people_lower = (df["cast"] + " " + df["crew"] + " " +
4                     df["production_companies"]).str.lower()
5      query_normalized_lower = query_normalized.lower()
6      exact_match = people_lower.str.contains(query_normalized_lower,
7                                              regex=False).astype(float)
8
9      fuzzy_match = people_lower.apply(lambda x:
10                                     fuzz.partial_ratio(query_normalized_lower, x) / 100.0)
11
12     people_score = np.maximum(exact_match, fuzzy_match)

```

```
8     return people_score
```

3.3.6 Funzione: calc_total_score

```
1     def calc_total_score(sentence_score_normalized, genres_score,
2                           best_title_score, best_people_score):
3         sentence_arr = np.array(sentence_score_normalized)
4         genres_arr = np.array(genres_score)
5         title_arr = np.array(best_title_score)
6         people_arr = np.array(best_people_score)
7
8         w_title = 0.3
9         w_people = 0.2
10        w_genres = 0.2
11        w_sentence = 0.3
12
13        if best_title_score.max() < 0.7:
14            w_people += 0.20
15            w_genres += 0.10
16            w_title = 0.0
17
18        if best_people_score.max() < 0.7:
19            w_sentence += w_people
20            w_people = 0.0
21        # Se il cast o la produzione
22        if best_people_score.max() >= 0.7:
23            w_sentence = 0.2
24            w_people = 0.3
25
26        total_score = (
27            (sentence_arr * w_sentence) +
28            (genres_arr * w_genres) +
29            (title_arr * w_title) +
30            (people_arr * w_people)
31        )
32        return total_score
```

3.3.7 Ricerca dei Film

```
1     def search_film():
2         user_query = input("Inserisci la tua query: ")
3
4         sentence_score = calc_sentence_score(user_query)
5
6         query_genres = find_genres(user_query, unique_genres)
7         genres_score = calc_genres_score(query_genres)
8
9         best_title_score = calc_title_score(user_query, df)
10
11        best_people_score = calc_people_score(user_query, df)
```

```

13     total_score = calc_total_score(sentence_score, genres_score,
14                                   best_title_score, best_people_score)
15     results = list(enumerate(total_score))
16     top_10 = sorted(results, key=lambda x: x[1], reverse=True)[:10]
17     index = [i[0] for i in top_10]
18     table = df.iloc[index][["title", "genres", "cast", "production_companies"]]
19     print("\nFilm Consigliati per te in base alla tua query:
        {}".format(user_query))
    return table

```

Film Consigliati per te in base alla tua query: dragon fantasy

	title	genres	cast	production_companies
292	Eragon	[fantasy, action, adventure, family]	Ed Speleers, Jeremy Irons, Sienna Guillory, Ro...	Ingenious Film Partners, Twentieth Century Fox...
160	How to Train Your Dragon 2	[fantasy, action, adventure, animation, comedy...	Jay Baruchel, Gerard Butler, Kristen Wiig, Jon...	DreamWorks Animation, Mad Hatter Entertainment
92	How to Train Your Dragon	[fantasy, adventure, animation, family]	Jay Baruchel, Gerard Butler, Craig Ferguson, A...	DreamWorks Animation, Vertigo Entertainment, M...
3013	Pete's Dragon	[adventure, family, fantasy]	Bryce Dallas Howard, Oakes Fegley, Wes Bentley...	Walt Disney Pictures, Walt Disney Studios Moti...
784	In the Name of the King: A Dungeon Siege Tale	[adventure, fantasy, action, drama]	Jason Statham, John Rhys-Davies, Ray Liotta, L...	Boll Kino Beteiligungs GmbH & Co. KG, Brightli...
98	The Hobbit: An Unexpected Journey	[adventure, fantasy, action]	Ian McKellen, Martin Freeman, Richard Armitage...	WingNut Films, New Line Cinema, Warner Bros. P...
71	The Mummy: Tomb of the Dragon Emperor	[adventure, action, fantasy]	Brendan Fraser, Jet Li, John Hannah, Maria Bel...	Universal Pictures, China Film Co-Production C...
1680	Savva. Heart of the Warrior	[fantasy, adventure, animation]	Maksim Chukharyov, Konstantin Khabenskiy, Milkh...	Art Pictures Studio
22	The Hobbit: The Desolation of Smaug	[adventure, fantasy]	Martin Freeman, Ian McKellen, Richard Armitage...	WingNut Films, New Line Cinema, Warner Bros. P...
899	Shrek	[adventure, animation, comedy, family, fantasy]	Mike Myers, Eddie Murphy, Cameron Diaz, John L...	DreamWorks SKG, Pacific Data Images (PDI), Dre...

4 Validazione

4.1 Funzione: get_top_k_titles

```

1  def get_top_k_titles(user_query, k=10):
2      query_genres = find_genres(user_query, unique_genres)
3      genres_score = calc_genres_score(query_genres)
4
5      query_pure = user_query.lower()
6      for gen in query_genres:
7          query_pure = query_pure.replace(gen, "").strip()
8          if not query_pure: query_pure = user_query.lower()
9
10     sentence_score = calc_sentence_score(user_query)
11     best_title_score = calc_title_score(query_pure, df)
12     best_people_score = calc_people_score(query_pure, df)
13
14     total_score = calc_total_score(sentence_score, genres_score,
15                                   best_title_score, best_people_score)
16     results = list(enumerate(total_score))
17     top_k = sorted(results, key=lambda x: x[1], reverse=True)[:k]
18
19     return [df.iloc[i[0]][["title"]] for i in top_k]

```

4.2 Validation Set

```

1  validation_set = [
2      # Query basilari

```

```

3   {
4       "query": "action robert downey jr.",
5       "expected": ["Iron Man", "The Avengers"]
6   },
7   {
8       "query": "animation jesse eisenberg",
9       "expected": ["Rio", "Rio 2"]
10  },
11  {
12      "query": "christopher nolan space",
13      "expected": ["Interstellar"]
14  },
15  {
16      "query": "pirates johnny dep",
17      "expected": ["Pirates of the Caribbean: The Curse of the Black Pearl"]
18  },
19  {
20      "query": "trein your dregon",
21      "expected": ["How to Train Your Dragon"]
22  },
23  {
24      "query": "zoe saldana action",
25      "expected": ["Guardians of the Galaxy", "Avatar"]
26  },
27
28  # Query miste con generi e attori
29  {
30      "query": "romance leonardo dicaprio",
31      "expected": ["Titanic", "The Great Gatsby"]
32  },
33  {
34      "query": "sci-fi keanu reeves",
35      "expected": ["The Matrix", "The Matrix Reloaded"]
36  },
37  {
38      "query": "action tom cruise",
39      "expected": ["Mission: Impossible", "Mission: Impossible II", "Mission:
40      Impossible III", "Edge of Tomorrow"]
41  },
42  {
43      "query": "kristen bell animation",
44      "expected": ["Frozen"]
45  },
46  {
47      "query": "action adventure fantasy sam worthington",
48      "expected": ["Avatar", "Clash of the Titans"]
49  },
50
51  # Query con registi e trame
52  {
53      "query": "christopher nolan dream",
54      "expected": ["Inception"]
55  },
56  {
57      "query": "quentin tarantino crime story",
58      "expected": ["Pulp Fiction", "Reservoir Dogs"]
59  },
60  {
61      "query": "steven spielberg dinosaur park",
        "expected": ["Jurassic Park"]

```



```

62     },
63     {
64         "query": "peter jackson ring fantasy",
65         "expected": ["The Lord of the Rings: The Fellowship of the Ring", "The
        Lord of the Rings: The Two Towers", "The Lord of the Rings: The Return
        of the King"]
66     },
67
68     # Query con brand e case di produzione
69     {
70         "query": "pixar animation",
71         "expected": ["Cars", "A Bug's Life"]
72     },
73     {
74         "query": "marvel studios action",
75         "expected": ["The Avengers", "Iron Man", "Guardians of the Galaxy"]
76     },
77     {
78         "query": "dreamworks animation",
79         "expected": ["Shrek", "Madagascar"]
80     },
81     {
82         "query": "warner batman",
83         "expected": ["The Dark Knight", "The Dark Knight Rises", "Batman Begins"]
84     },
85
86     # Query semantiche
87     {
88         "query": "space travel alien planet",
89         "expected": ["Avatar", "Interstellar", "Prometheus"]
90     },
91     {
92         "query": "wizard school boy",
93         "expected": ["Harry Potter and the Philosopher's Stone"]
94     },
95     {
96         "query": "world war II nazist",
97         "expected": ["Schindler's List", "Inglourious Basterds"]
98     },
99     {
100        "query": "artificial intelligence robot",
101        "expected": ["I, Robot", "Robocop", "Ex Machina"]
102    },
103    {
104        "query": "disney lion kingdom",
105        "expected": ["The Lion King"]
106    },
107    {
108        "query": "science fiction dinosaur come back",
109        "expected": ["Jurassic Park"]
110    },
111
112    # Query refusi
113    {
114        "query": "iron men",
115        "expected": ["Iron Man"]
116    },
117    {
118        "query": "quentin tarantno",
119        "expected": ["Pulp Fiction", "Inglourious Basterds"]

```

```

120     },
121     {
122         "query": "atroboy",
123         "expected": ["Astroboy"]
124     },
125     {
126         "query": "christofer nolan",
127         "expected": ["Inception", "Interstellar", "The Dark Knight"]
128     },
129     {
130         "query": "johnny depp wanderlund",
131         "expected": ["Alice in Wonderland"]
132     }
133 ]

```

4.3 Funzione: run_validation

```

1  def run_validation(test_set, k=10):
2      total_expected_movies = 0
3      hits = 0
4      reciprocal_ranks = []
5
6      print("Ricerca in corso...\n" + "-"*50)
7
8      for i, test in enumerate(test_set):
9          query = test["query"]
10         expected_list = test["expected"]
11         predictions = get_top_k_titles(query, k=k)
12
13         print(f"Test #{i+1} | Query: '{query}'")
14         print(f"Raccomandati Top 3: {predictions[:3]}")
15
16         for expected_movie in expected_list:
17             total_expected_movies += 1
18             if expected_movie in predictions:
19                 hits += 1
20                 # Calcolo del rank per l'MRR (posizione del film nei risultati)
21                 rank = predictions.index(expected_movie) + 1
22                 reciprocal_ranks.append(1.0 / rank)
23                 print(f"Trovato: '{expected_movie}' in posizione {rank}")
24             else:
25                 reciprocal_ranks.append(0.0)
26                 print(f"Mancato: '{expected_movie}' non presente nei top {k}")
27         print("-"*50)

```